HireLoop Project Requirements Analysis

Database: Kevin Dong and Avi Maheshwari
Frontend: Eshaan Salvi and Charan Sama
API layer: Katelyn Doucette and Rishi Chawla

Github Link: https://github.com/RishiChawla1/CS3704-HireLoop.git

Database Use Cases:

Use case DUC1: Response is sent to a job query/application

Primary actor: Company

Stakeholders and interests: Company, Applicant. Company wants to inform applicants of their application status, applicant wants to know the result of the application.

Preconditions: Job query has at least one application.

Success guarantee: Job posting in the database will be tagged with the response type (rejection, acceptance, other, etc)

Main success scenario:

- Company sets a particular application as “not interested” on the web page
- Request is sent to the API, which then validates and forwards it to the database
- Database receives a request to update the tag for an application ID
- The database replaces the tag with the new tag
- The database returns to the API that it successfully updated the tag of that application
- API sees that the database succeeded, so tells the front end
- Front-end displays to the company that that application was updated

Extensions:

1. If the applicant no longer has an account, the application is discarded
2. If the specific application does not exist, the database informs the API that an error occurred so that it can handle what went wrong.

Special requirements: None

Technology and data variation list: Relevant databases.

Use case DUC2: Data about applicants is organized by tags

Primary actor: Company

Stakeholders and interests: Company, Company wants to easily track data about applicants.

Preconditions: Company has applicants.

Success guarantee: Should work so long as data is tagged correctly.

Main success scenario:

- A company posts an open job vacancy.

- Applications are filled and submitted for the vacancy.
- The company then organizing all the applicants in an order determined by the the tag requested
- The page then displayed with the ordered data

Extensions:

1. Not all applicants have the tag used, NULL values will be placed last in the order.
2. Two different applicants have the same tag when ordering the applicants, then the system should order secondarily by name and then date of birth.

Special requirements: Data tagging.

Technology and data variation list: Database of applicants with options for tags.

Use case DUC3: A user's information is autofilled on an application

---

Primary actor: Applicant

Stakeholders and interests: Applicants, companies. Applicants want to fill out applications faster, companies want high quality applications.

Preconditions: Applicant agrees to autofill, applicant has relevant information to autofill.

Success guarantee: The webpage's application form has all the relevant information filled in automatically on the form.

Main success scenario:

- Applicant begins an application.
- A popup appears requesting if the applicant would agree to autofilling some information.
- After loading the fields which could be autofilled are now populated with the correct information.

Extensions:

1. Applicant denies use of autofill, then autofill should not execute and leave all the fields blank.
2. The application has no data that can be autofilled, then the fields should stay blank and the user will be notified by a dialog that the autofill did not fill in any fields.
3. After loading, the wrong information is some field, the application should catch this when filling out the form. The system will also notify the user if any information is mismatched and request the applicant verify all the information is correct.

Special requirements: None

Technology and data variation list: User profile with autofill information.

Use case DUC4: The status of an application is tracked

Primary actor: Applicant

Stakeholders and interests: Applicant, applicant wishes to know if they have been rejected or accepted as soon as possible.

Preconditions: Applicant has an application that can be tracked.

Success guarantee: The applicant can see in their webpage that the application was accepted or denied.

Main success scenario:

- The applicant creates and sends in the application,
- The company sees and elects to conduct a round of interviews with the applicant, and creates a message response conveying such with the application status now being changed to accepted.
- The message is sent to the applicant and is read.
- The application status tracked updates to show the .

Extensions:

1. System fails to email the applicant, the system should automatically show messages that have not been sent and request for the user to resend the message.
2. The webpage displays the wrong status. The applicant can manually change the status of applications to be correct.

Special requirements: None

Technology and data variation list: Application database per user

Use case DUC5: Be able to store messages between users

Primary actor: Applicant, Company

Stakeholders and interests: Company, Applicant. Applicants want to communicate with companies about jobs, companies want to respond to applicants and hire them.

Preconditions: Both users have accounts and a conversation exists between them.

Success guarantee: Messages between users are successfully stored and retrievable by future queries to the database

Main success scenario:

- A sender user writes out a message on the front-end, and hits send
- The message is sent to the API, along with a flag that there are no attachments, and a timestamp
- This message is validated, and then a query is sent to the database to store it
- The database adds the message to its records with tags indicating the timestamp, sender, and recipient.

Extensions:

1. If the recipient account no longer exists, or the user doesn't have permissions to send it, this will get blocked by the API layer or front-end and display an error to the user
2. If the database fails to save the message, report an error to the API.
3. The API would store an attachment in the server file system and send the database a link to the relevant attachment.

Special requirements: None

Technology and data variation list: Message DB sender ID, receiver ID, message content, and timestamp;.

Use case DUC6: Be able to store a user's bookmarked job postings

---

Primary actor: Applicant

Stakeholders and interests: Applicant, Company. Applicants want to save job postings they are interested in, companies want applicants to return to their postings.

Preconditions: Job posting exists and the applicant has an account.

Success guarantee: Bookmarked job posting is successfully stored and retrievable for the user.

Main success scenario: Applicant views a job posting and selects the bookmark option. The backend stores the job posting in the applicant's bookmarked list. The applicant can view their bookmarked job postings on their profile.

- The applicant finds a job listing that they are interested in.
- The applicant clicks a "bookmark" icon on the job posting.
- The API receives the request from the frontend and validates the request
- The API tells the database to add it to the bookmarks for the user
- The database writes it to the correct place
- The database returns successful
- The API forwards the success to the frontend, and a banner stating "job successfully saved" is displayed temporarily.

Extensions:

1. If the job fails to save, an alternate banner is displayed stating that something went wrong with job saving
2. If the job listing is then removed, it is removed from the applicant's bookmarks
3. If the API is provided with an invalid job listing to bookmark, it should detect that and return an error code (404 not found).

Special requirements: This should be very quick, doable in under half a second. This is because it's a common and lightweight operation so thus should feel responsive to the user.

Technology and data variation list: Bookmarked column with job posting IDs.

Use case DUC7: Be able to store which connections a user has

Primary actor: Applicant

Stakeholders and interests: Applicants, Companies. Applicants want to connect with users and companies for networking and opportunities.

Preconditions: Both users have accounts on the platform.

Success guarantee: Connections between users are successfully stored and retrievable.

Main success scenario: A user sends a connection request to another user. The receiving user accepts the request. The system stores the connection between the two users in the database and both users can view the connection in their network list.

- A user sends a connection request to another user
- This connection request is routed to the API which then adds that outgoing connection request to a list that the other user can see
- The other user accepts the connection request
- The database adds the two users to each other's connections lists

Extensions:

1. The connection request gets declined. The connection request is removed from the user's outgoing connection requests
2. If the user isn't accepting connections, the front-end greys out the button and the API refuses to forward the request to the database
3. If the connection request is outgoing and either account is deleted or otherwise disabled, the connection request is deleted
4. The connection request times out after 1 month
5. If the connection request fails to get through, or doesn't get stored in the database, it is displayed as not working to the user.

   Special requirements: Connections and connection requests should be stored contiguously or easily accessible. Connection requests are often requested in batches and we want to retrieve them quickly.

   Technology and data variation list: Connection database storing connections between 2 user IDs.

Use case DUC8: Be able to provide a list of users based on specific tags

---

Primary actor: Company

Stakeholders and interests: Company, Applicants. Companies want to find applicants with certain tags, applicants want to be found by companies based on their tags.

Preconditions: Applicants exist in the system and have tags on their profiles.

Success guarantee: The system returns a list of users that match a tag

Main success scenario: A company selects a tag. The system queries the database for users associated with that tag. The system returns and displays a list of matching applicants.

Extensions:

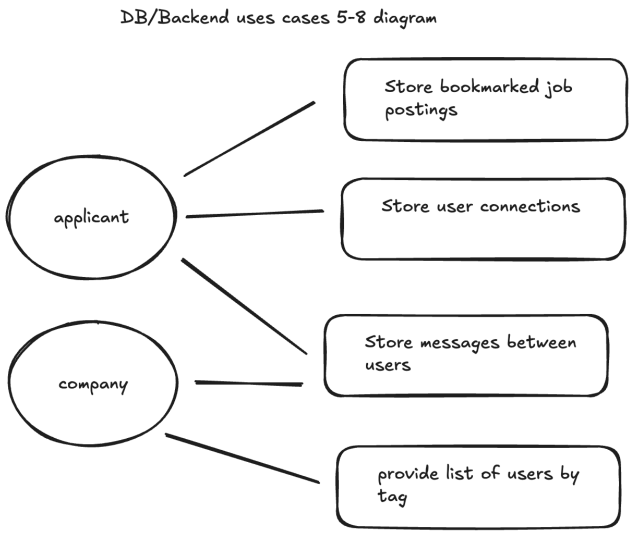
1. No users match the selected tag
   a. Backend receives request with tag filter
   b. Backend validates request format
   c. Backend executes query against database index
   d. Query returns 0 matching records
   e. Backend returns HTTP 200 OK
   f. Backend logs as INFO
   g. Use case ends successfully
2. Tag does not exist in the system
   a. Backend receives request containing a tag value
   b. Backend validates the tag
   c. Backend determines the tag is invalid
   d. Backend returns HTTP 400 Bad Request
   e. Backend logs as WARN including the tag value
   f. Use case ends
3. Database query fails
   a. Backend receives and validate the search request
   b. Backend attempts database search index query
   c. Database service returns an error

d. Backend applies a failure strategy
    i. Retry the query up to N times
e. If retry succeeds
    i. Return HTTP 503 Service Unavailable
f. Backend logs as ERROR with stack race
g. Use case ends

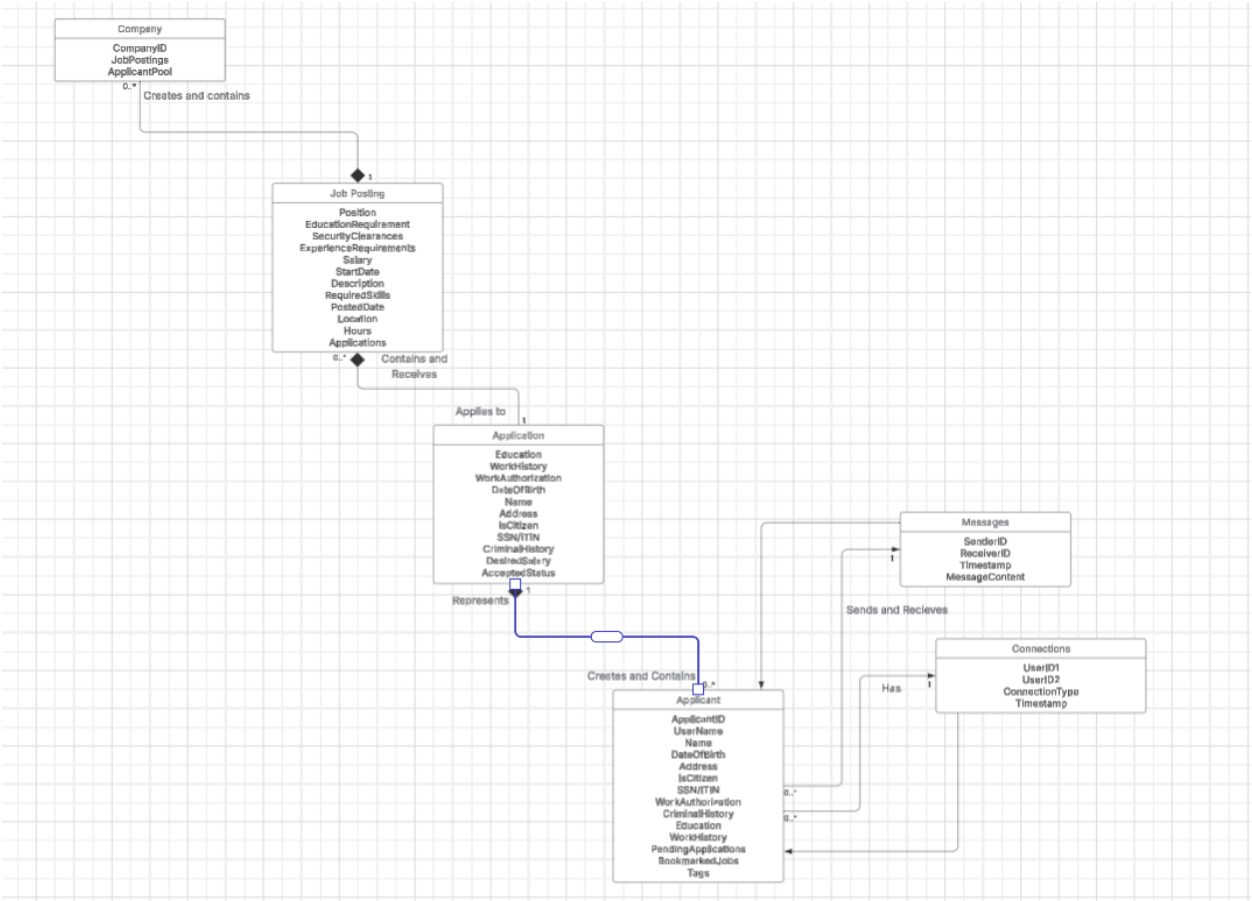
Special requirements: The database needs to be efficient at handling sparse data for this purpose.

Technology and data variation list: User database has tags to each user profile.

Backend Use Case Diagram:



Backend ULM Diagram:



Frontend

Use case UUC1: Search for job posting

Primary actor: Job Seeker
Stakeholders and interests: Candidate, HireLoop, Employers/Recruiters
Preconditions:

- Candidates can access the HireLoop application
- The system has job postings stores and searchable

Success guarantee: Matching job postings or a “no results” message are deployed to the candidate.

Main success scenario:

- The candidate navigates to the job search page and sees a search bar.
- The candidate enters any keywords or tags.
- The candidate submits the search.

- The system sends the query to the backend search API.
- The system receives a list of matching job postings.
- The system displays results in a readable list.
- The candidate selects a job posting

Extensions:

1. Candidate submits an empty query
    a. System detects that the search field is empty
    b. System displays validation message “Please enter at least one keyword”.
    c. System highlights the search input field.
    d. Flow returns to Step 2 so the candidate can modify the query.
2. Search API timeout/error
    a. System detects failure to reach the backend server
    b. System displays message “Search failed. Please try again”.
    c. System logs the error for debugging purposes.
    d. System preserves the candidate’s search query.
    e. Candidate selects “Retry”.
    f. System responds to the request and resumes at Step 4.
    g. If retry fails multiple times, the system suggests refreshing the page.
3. No results found
    a. System determines that the returned result set is empty.
    b. System displays message “No jobs matched your search”.
    c. System suggests broadening the search terms.
    d. Flow returns to Step 2.

Special requirements: None

Technology and data variation list: Search may accept keywords and tags depending on API support.

Use case UUC2: View job posting details

Primary actor: Candidate

Stakeholders and interests: Candidate, HireLoop, Employers/Recruiters

Preconditions:

- Candidates have performed a search or have a direct link to the job posting.
- The job posting exists in the system.

Success guarantee: The job posting detail page is displayed with full posting info.

Main success scenario:

- The candidate clicks a job posting from search results.
- The system requests the posting from the backend.
- The system receives the posting details.
- The system renders the job posting page in the UI.
- The candidate views details and may choose next actions like bookmarking the posting or applying to the job.

Extensions:

1. Posting not found
    a. Systems detect that the job ID is invalid or the posting has been removed.
    b. System displays message “The job posting is no longer available”.
    c. System provides option to return to the search results page.
    d. Candidate selects “Back to Results”.
2. Backend service unavailable
    a. System fails to retrieve job posting data.
    b. System displays error message with “Retry” option.
    c. System logs the error.
    d. If the candidate selects “Retry”, the system repeats Step 2.
3. Large description
    a. System detects description exceeds predefined display length.
    b. System collapses part of the description.
    c. Candidate may expand the section to view full content.

Special requirements: None

Technology and data variation list: The job detail may include tags, requirements, and location, depending on stored fields.

Use case UUC3: Application Statuses

Primary actor: Candidate

Stakeholders and interests: Candidate, HireLoop

Preconditions:

- The candidate is logged in and has a profile.
- The candidate has at least one tracked application or the system supports an empty dashboard state.

Success guarantee: The candidate sees a dashboard of their applications and current statuses.

Main success scenario:

- The candidate navigates to the “My Applications” page.
- The system requests the candidate’s applications and statuses from the backend.
- The system receives the list of applications with status info.
- The system displays the dashboard.
- The candidate clicks an application to view its details.

Extensions:

1. Candidate not authenticated
    a. System detects that candidate session is invalid or expired.
    b. System redirects candidate to login page.
    c. Candidate logs in successfully.
    d. System returns candidate to the “Application Status” page.
    e. Flow resumes at Step 3.
2. Candidate has no applications
    a. System determines candidate has no stored applications.

    b. System displays message “You have not applied to any jobs yet”.
    c. System provides link to job search page.
3. Backend database failure
    a. System fails to retrieve application records.
    b. System displays temporary error messages.
    c. System logs error.
    d. Candidate may retry loading the page.

Special requirements: None
Technology and data variation list: Status values like “Applied”, “Interview”, “Accepted”, “Rejected” may vary.

Use case UUC4: Direct Messaging

Primary actor: Candidate, Recruiter/Employer
Stakeholders and interests: Candidate, HireLoop, Employers/Recruiters
Preconditions:

- The user is logged in.
- The user has access to the messaging feature and valid recipients exist.

Success guarantee: A message is sent and appears in the conversation thread.
Main success scenario:

- The user opens the “Messages” page.
- The user selects an existing conversation or chooses a recipient.
- The user types a message and presses “Send”.
    - The user optionally adds an attachment first
- The system sends the message to the messaging API.
- The system receives confirmation of the message being sent.
- The system displays the sent message in the thread.

Extensions: Empty query, messaging API error, unauthorized recipient.

1. The message was empty or only contains whitespace
    a. Message send button is disabled
2. If there’s a backend error, or the request is malformed, display “the message could not be send”
3. If the recipient can’t be messaged (messages disabled, etc) state that the message could not be delivered due to (reason)
4. If the attachment is too large, refuse to send the message due to the size of the attachment
5. If the message is too long, state that the message is too long.

Special requirements: None
Technology and data variation list: Real time update may use polling/websockets depending on the implementation.

Use case UUC5: Sign up page, create account

Primary actor: Candidate
Stakeholders and interests: Candidate, HireLoop, Employers/Recruiters
Preconditions:
- Candidate has access to the app and provides the required sign-up information

Success guarantee: User account is created, and the candidate can log in
Main success scenario:
- Candidate opens the sign-up page
- The user enters their name, email address, password, and confirms the password
- Submit the form to make the account
- Sends the user an email for confirmation
- User confirms via a unique link in their email
- System sends sign-up request to the backend API
- System receives confirmation and the user session/token.
- The user will now have an account and can access anything on the site

Extensions: Email already exists, weak password, backend error/time-out.

1. There's something wrong with the email
    a. If the email is already associated with an account, state that an account already exists in the email (don't state on the front end to prevent credential stuffing attacks)
    b. If the email doesn't exist, display that it doesn't exist and prompt the user to re-enter it.
2. If the provided password is insecure (no capital letters, and no number or symbol), refuse to make the account due to an insecure password
3. If there's an internal server error, state to come back later and that the account couldn't be created

Special requirements: The signup needs to be done securely (with E2E encryption via HTTPS at least)
Technology and data variation list: Use email/password, and would require email verification depending on backend support.

Use case UUC6: Apply to a Job Posting

Primary actor: Candidate
Stakeholders and interests: Candidate, HireLoop
Preconditions:
- Candidate is logged in
- Job posting exists.

Success guarantee: The application is submitted, and the candidate sees confirmation.
Main success scenario:
- Candidate is on a job posting page
- Candidate clicks apply

- The system displays the application form
- Candidate fills the required fields and submits the application
- The system sends the application to the backend application API
- System receives success confirmation and shows “Application submitted.”

Extensions: Candidate not logged in, missing required fields, and upload error.

Candidate not logged in

1. Candidate selects “Apply.”
2. The system detects that the candidate is not authenticated
3. System redirects the candidate to the login page
4. Candidate logs in successfully
5. System redirects the candidate back to the job posting page

Missing required fields

1. Candidate submits application form with incomplete fields
2. The system detects missing required information
3. System highlights the missing fields
4. System displays message “Please complete all required fields.”

Special requirements: None

Technology and data variation list: Resume upload would be a file upload and can be used on the page, or it can be typed into if needed.

Use case UUC7: Filter Job Search Results

Primary actor: Candidate

Stakeholders and interests: Candidate, HireLoop

Preconditions:

- The candidate is on the job search results page, and job postings exist

Success guarantee: Filtered job postings matching the selected criteria are displayed.

Main success scenario:

- Candidate performs a job search and receives results
- Candidate selects one or more filters
- System sends the filter parameters to the backend search API
- System receives filtered results and updates the page
- Page is displayed as a continuous endless scroll, and more jobs are displayed as user scrolls through

Extensions: No results match the filters, timeout/error, invalid filter value

1. If no results match the filters, display “no results”
2. If the request timed out, urge the user to try again
    a. If there’s no internet, specify they should reconnect first
    b. If it’s an internal server error, specify that they should try again later
3. If a filter value is invalid, state that an invalid filter value was detected and can’t be fulfilled before firing the request

Special requirements: None

Technology and data variation list: Filters may include tags, job type, location, or other filterable fields that the candidate could select.

Use case UUC8: Edit Profile for Users

Primary actor: Candidate
Stakeholders and interests: Candidate, HireLoop
Preconditions:
- Candidate is logged in and has a profile.

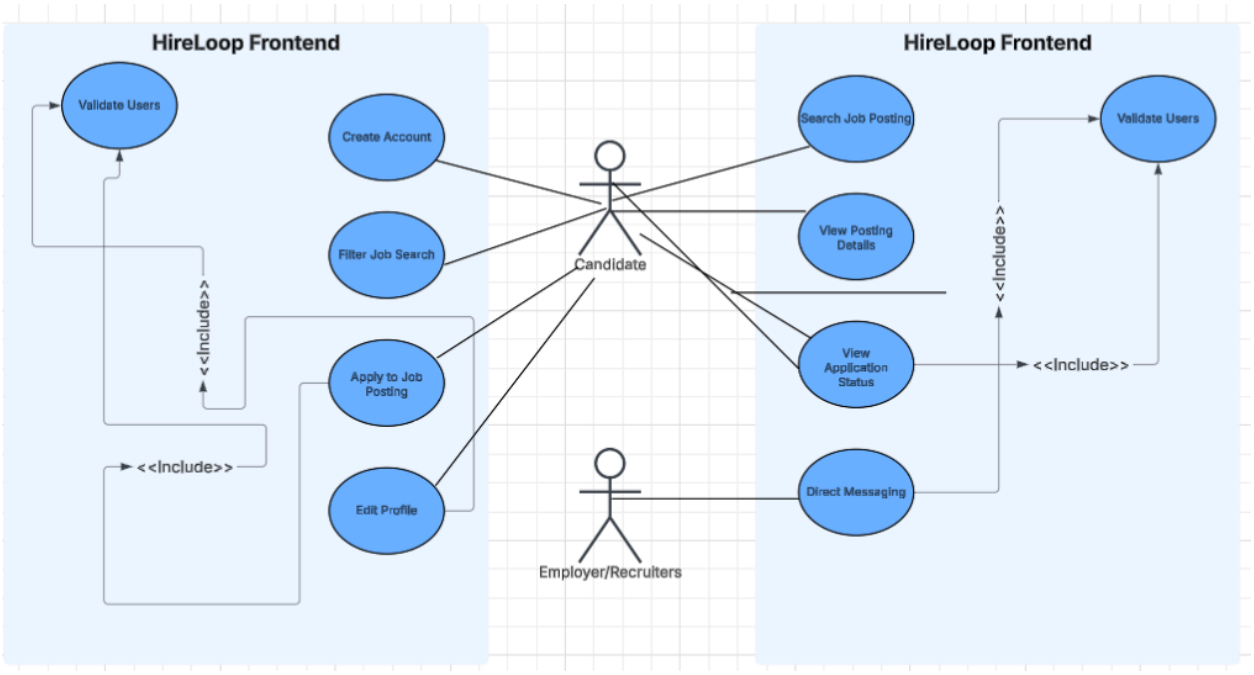
Success guarantee: Profile changes are saved and reflected in the UI
Main success scenario:
- Candidate navigates to Profile
- Selects Edit Profile
- Candidate updates fields and clicks Save
- System sends updated profile data to the backend profile API
- System receives success confirmation and displays the updated profile

Extensions:

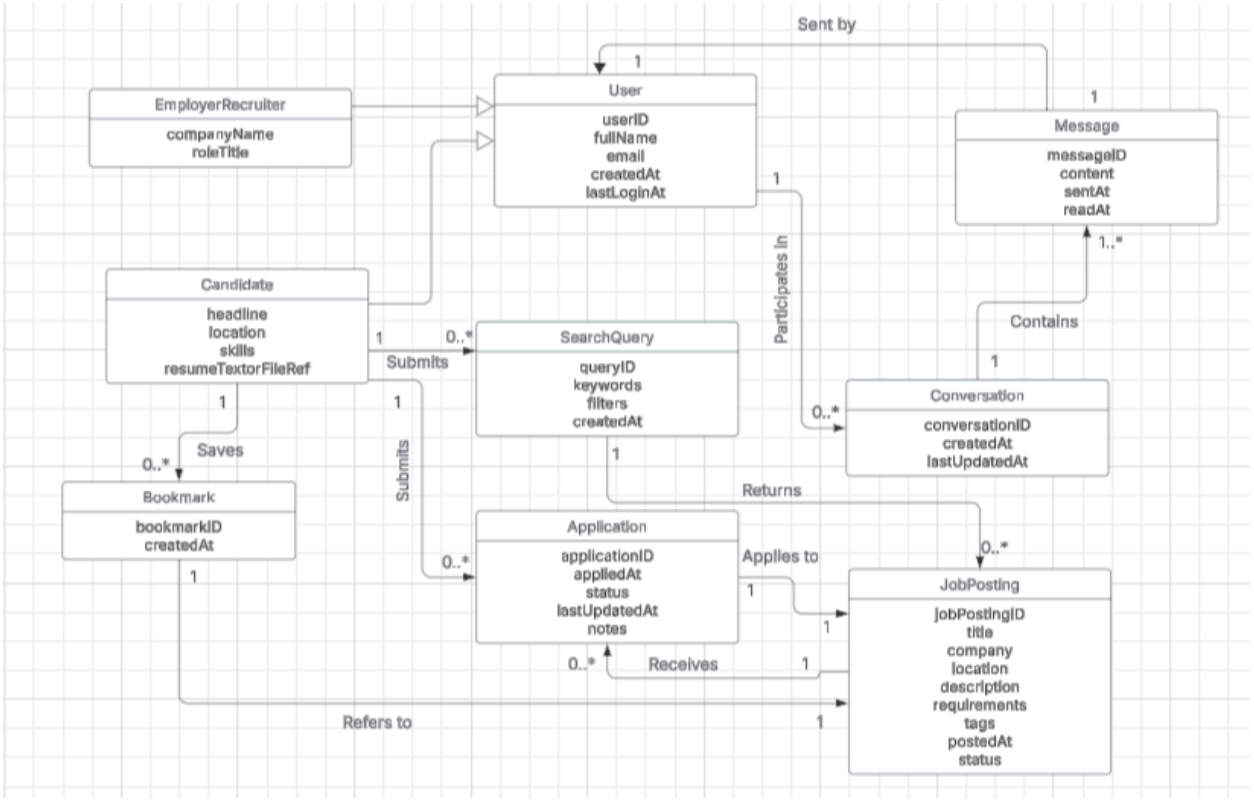
1. Invalid profile field
    a. All invalid fields should be highlighted in red with an error message of why they are invalid
2. Resume upload fails
    a. If the failure is due to network errors, display that it failed due to lack of internet
    b. If the failure is due to an internal server error, display that we are sorry and will be back soon
    c. If the resume is an invalid format, reject the resume and display that it's an invalid resume
3. If there is a general backend error, display that there is a problem to the user.

Special requirements: None
Technology and data variation list: resume may be a file upload, and there can be tags and certain profile requirements that should be filled out by the user.

Use Case Diagram:



Frontend UML Diagram:



Frontend Supplementary Specifications

- **Usability of the frontend application**, the user should be able to complete common actions like searching for jobs or applying to jobs in a short amount of time. The pages should load normally in network conditions.
- **UI accessibility and consistency**, all of the pages must maintain a consistent layout, color scheme, and navigation throughout the site to provide a more consistent user experience.

API:

Use case AUC1: Create Job Opening

Primary actor: Employer/Recruiter

Stakeholders and interests: The employer and the applicant are the main stakeholders. Employers are interested in posting job applications to fill job positions, and applicants are interested in relevant jobs to apply to.

Preconditions: The employer has a verified account that is authorized to post jobs to the app.

Success guarantee: Job opening is successfully stored in the system and is visible to all applicants

Main success scenario:

- Employer submits a new job opening through the API with a title, description, requirements, and tags
- API validates the request
- API stores the new job in the database
- Applicants are now able to see the new job posting

Extensions: If required fields are missing in the posting, like job description or requirements, then the API rejects the request and sends an error message

Special requirements: None

Technology and data variation list: Any relevant database

Use case AUC2: Get AI feedback on resume

Primary actor: Candidate, relevant AI company

Stakeholders and interests: The candidate is the main stakeholder. The company whose AI service is being used is also a stakeholder as they will be providing the model that gives the feedback.

Preconditions: The employee has an account and has uploaded their resume to their account, or has provided it to the service.

Success guarantee: A list of things to change about the wording of the resume depending on what the candidate requests.

Main success scenario:

- User uploads resume to page, sends request for feedback by pressing a submit button.
- API validates request
- API uses a library to pull out the text of the PDF.
- API forwards that request to an AI service and awaits a response.
- Upon hearing back, the API extracts the AI response and forwards it back to the user.

Extensions:

1. Request is malformed or otherwise invalid - returns an error code
2. PDF is malformed - returns an error code.
3. AI endpoint fails to respond properly
    a. No response at all - should timeout after 30 seconds with no communication.
    b. Error code - API should forward that error message to the user
4. Library fails to parse the PDF - Return an error message indicating that the PDF could not be parsed

Special requirements: None

Technology and data variation list: An AI service must be partnered with to implement this feature. A library for parsing the PDF is also necessary.

Use case AUC3: Edit Job Opening

Primary actor: Employer/Recruiter

Stakeholders and interests: The employer and the applicant are the main stakeholders. Employers are interested in keeping job postings up to date and accurate, while applicants want to see current job information.

Preconditions: The employer has a verified account and owns the job posting that is being edited.

Success guarantee: The job opening is successfully updated with the new information in the system.

Main success scenario:

- Employer submits an edit request through the API with the job posting ID and updated information and fields
- API validates the edit request is from the employer that owns the job posting
- API updates the job postings fields
- API applies the changes to the job posting to the system.
- Updated job posting is now available to the applicants

Extensions:

1) User attempts to modify posting they don’t have access to
    a) If they are logged in, respond with an error that they aren’t logged in
    b) If they are logged in but don’t have access, respond that they don’t have that permission
2) If the request is invalid, respond that it is invalid.

3) If one or more fields are missing, don't do anything and respond that that field is missing.
   Special requirements: None
   Technology and data variation list: Any relevant database

Use case AUC4: Submit a job application

Primary actor: Applicant

Stakeholders and interests: The employer and the applicant are the main stakeholders. Employers are interested in receiving job applications to review, and applicants are interested in sending their job applications to recruiters to get hired.

Preconditions: The applicant is authenticated and the job posting they are submitting an application to exists and hasn't expired.

Success guarantee: The application is successfully stored in the system and the employer receives the application.

Main success scenario:

- Applicant submits the application through the API with the job posting ID and required application material such as resume and cover letter
- API verifies that the applicant is authenticated and the job posting exists
- API validates that all required fields in the application are filled out
- API stores the application in the database
- Employer is sent the application and notified
- Applicants application is changed to "Applied" and can now track application status

Extensions:

1) If the job posting doesn't exist or is no longer open, the API should reject the request and inform the applicant that the job posting doesn't exist.
2) If required fields haven't been filled out, the API should send an error message and notify the applicant as to what hasn't been filled out.
3) If the applicant has already applied to the job before, the API should reject the submission and inform the applicant that it's a duplicate application.
   Special requirements: None
   Technology and data variation list: Any relevant database

Use case AUC5: Get status of application

Primary actor: Applicant

Stakeholders and interests: Applicant

Preconditions: The applicant is authenticated

Success guarantee: Returns a code indicating whether the application was accepted, rejected, or other.

Main success scenario:

- Applicant views a job posting that they previously applied to. Front-end sends request to the API

- API receives a unique identifier for the relevant job and applicant. The API validates the request.
- API gets the list of postings the user has applied to. The API then finds the relevant job in question, and checks the status.
- API responds with the resulting status.

Extensions:

1) If the user has not applied to the job, the API should respond with that so the front-end knows.
2) If the candidate has an outstanding application
    a) If the user has been rejected, it responds with that
    b) If the user has been accepted, it responds with that.
3) If there was an error in retrieving the data from the database, the API responds with an error code.
4) If the request is invalid, the API responds with an error code.

Special requirements: This request needs to be very quick, with average response time (excluding communication time between the database and front-end) of less than 1ms as this request happens a lot.

Technology and data variation list: Any relevant database. We also need to hold onto user application statuses for at least 6 months.

Use case AUC6: Update applicant profile picture

Primary actor: Applicant

Stakeholders and interests: Applicant, all users that can see the applicant.

Preconditions: The applicant is authenticated

Success guarantee: Applicant profile picture was successfully changed and a response code indicating so was returned

Main success scenario:

- Applicant selects a new profile picture for their page and hits “save”
- API receives the request for a page update.
- API validates the request
- API saves the picture to a server’s file system, then update the internal link to the picture if needed
- If all succeeds, the API responds with a success code.
- API deletes the original file.

Extensions:

1. Invalid image file
    a. If the user sends a file of an invalid format, it fails with an error code.
    b. If the image cannot be decoded according to the specified format, it fails with an error code.
    c. If the image is too large, it fails with an error code,
2. Invalid request (malformed header, etc.) fails with an error code.
3. Database failure

    a. If the database fails to save the new link to the picture, it should roll back the changes to the file system.
4. Filesystem failure, or inability to save the file to disk, should return an internal server error.

Special requirements: This request can take its time, but it needs to be reliable. >99.9% of failures to save the file to disk, or update the database, should result in a safe rollback.

Technology and data variation list: Any relevant database, as well as the file system being used to store the image data.

Use case AUC7: User logs in

Primary actor: Any user, including a company, applicant, recruiter, or other.

Stakeholders and interests: All users of HireLoop, and HireLoop itself.

Preconditions: User is on the site and currently logged out.

Success guarantee: User's browser receives a cookie indicating that they are now logged in

Main success scenario:

- The user submits a username and password.
- The API receives that username and password. The API checks that username against the database to find a match
- The database responds, indicating that the user exists and provides the salt for that user.
- The API hashes that password with the salt.
- The hashed password is queried against the hash of the user's password hash stored in a separate database.
- If it matches, the API creates a cookie and sends it with a success code.

Extensions:

1. Username or password is incorrect
    a. Username is incorrect: API responds with generic login failure
    b. Password is incorrect: API responds with generic login failure
        i. The failure message has to be identical between these to avoid credential stuffing attacks
2. Request is malformed - API responds with malformed request error.
3. The user has attempted to log in too many times already - reject with a notice that too many attempts have been tried.
    a. Necessary to prevent brute forcing

Special requirements: The hashing algorithm must be a well known hashing algorithm that is known to be cryptographically secure. The login system should also be stateless to be more secure. Login cookies should expire after 24 hours. Should be a separate database for storing passwords as well from everything else for security.

Technology and data variation list:

- A secure database for storing the passwords
- E2E encryption

- Requires a certificate from a trusted certificate authority

Use case AUC8: Delete Job Posting

Primary actor: Employer/Recruiter

Stakeholders and interests: The employer and the applicant are the main stakeholders. Employers are interested in removing job postings when the job is filled or the opening is being canceled, and applicants need to know when a job posting is no longer available.

Preconditions: The employer has a verified account and owns the job posting that is being deleted

Success guarantee: The job opening is successfully deleted from the system and can no longer be seen by applicants

Main success scenario:

- Employer submits a delete request to the API with the job posting ID of the job being deleted
- API verifies that the employer deleting the job posting owns the posting
- API removes the job from the database
- Job posting is no longer visible to applicants, but all applications that have been submitted are viewable by those who submitted it.

Extensions:

1) User attempts to modify posting they don't have access to
    a) If they are logged in, respond with an error that they aren't logged in
    b) If they are logged in but don't have access, respond that they don't have that permission
2) If the request is invalid, respond that it is invalid.

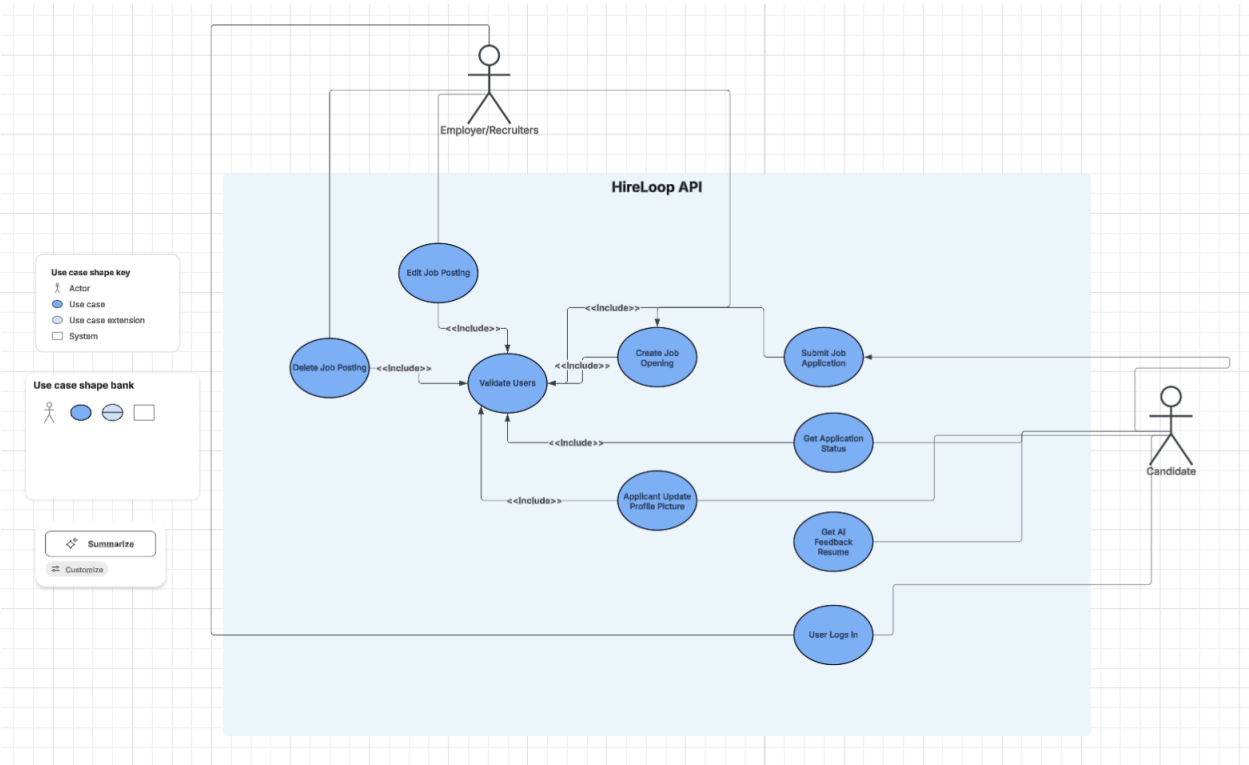
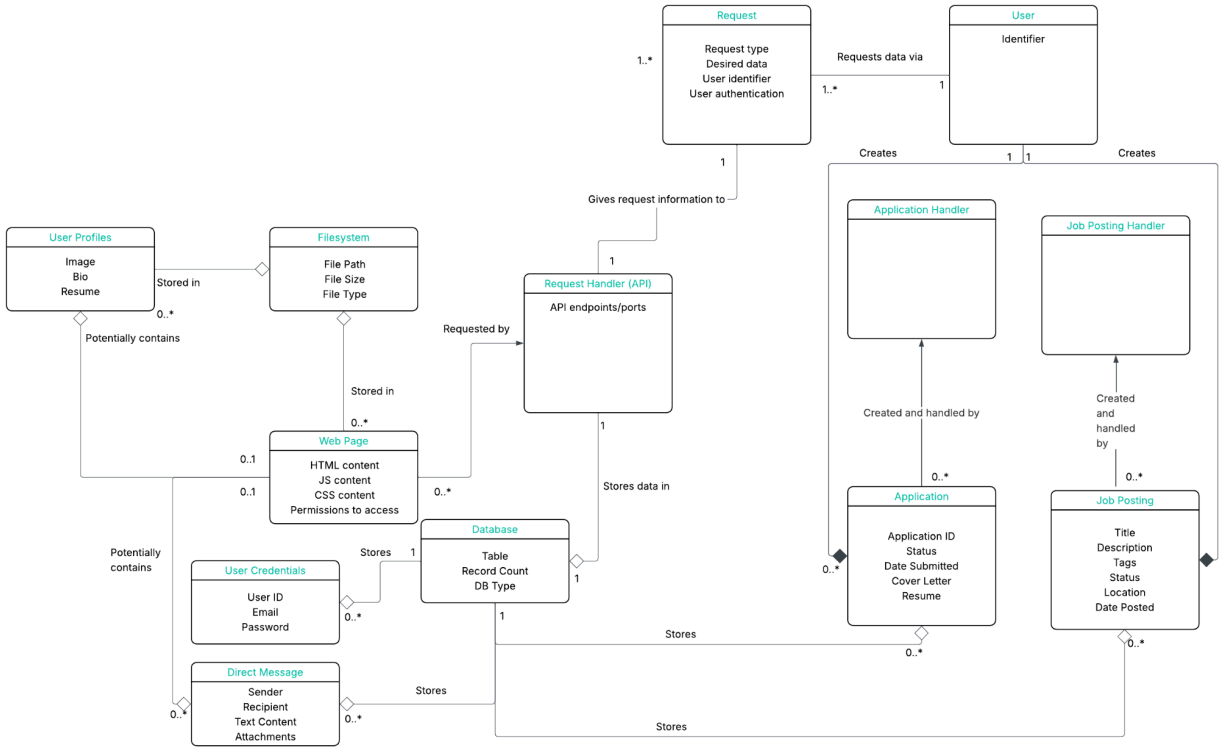
Special requirements: User applications should be visible for at least 6 months.

Technology and data variation list: Any relevant database

API Supplemental Specifications:

1) **Security** - The authentication system should be secure against all common cybersecurity threats.
2) **Responsive** - Web pages should be server-side-rendered by the API in order to improve responsiveness. The API itself should be able to respond in under a millisecond to the most common requests.
3) **Statelessness** - The API should be stateless.

Use Case Diagram:



Conceptual Class Diagram: